

AN2DL - First Challenge Report

Gradient Gang

Paolo Ginefra, Martina Missana, Lorenzo Petteuuzzo, Matteo Romilio Pasqual

paologinefra, martinamissana1, petteuuzzolorenzo, matteopasqual02

278153, 274235, 273597, 274006

November 16, 2025

1 Introduction

This project addresses the first challenge of the *Artificial Neural Networks and Deep Learning* course (A.Y. 2025–2026), which focuses on *multivariate time series* classification. The task consists of predicting pain intensity levels from a collection of temporal signals.

The dataset is composed of n multivariate time series of fixed temporal length $T = 160$. For each sample $i \in \{1, \dots, n\}$, the raw input is a tensor $X_i \in \mathbf{R}^{F \times T}$, where $F = 38$ denotes the total number of feature channels. These channels include: 4 survey-derived channels, 31 biomechanical channels, and 3 global features. Each multivariate time series has a categorical label $y_i \in \{0, 1, 2\}$, corresponding to *No Pain*, *Low Pain*, and *High Pain* so the goal is to learn $f_\theta : \mathbf{R}^{F \times T} \rightarrow \{0, 1, 2\}$. The challenge requires the development of a neural network where we need to capture both short- and long-range temporal dependencies while maximizing the F1-score.

2 Problem Analysis

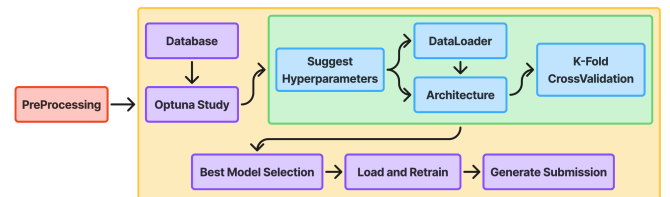
The problem presents several key challenges. The model must capture both long-range temporal dependencies and cross-channel interactions, while integrating heterogeneous features. The limited dataset size and class imbalance increases the risk of overfitting and the bias predictions toward frequent labels, requiring regularization, careful hyperparameter tuning, and robust cross-validation.

Our approach is based on several assumptions.

Pain levels are primarily reflected in temporal dynamics, including trends, variability, and motion irregularities, which may occur at multiple temporal scales from slow posture drifts to fast joint oscillations, motivating the use of convolutional and multiscale encoders. Global static features, such as subject-specific metadata, provide priors that influence pain prediction and should be integrated with dynamic signals. Dimensionality reduction techniques, as PCA or POD, can capture dominant modes, reduce redundancy, and improve generalization.

3 Methods

We have built a complete pipeline, from preprocessing that transforms raw time series into structured model inputs, to the end-to-end training framework responsible for model construction, hyperparameter optimization, and validation.



3.1 Pre-processing

Let the raw dataset consist of $X \in \mathbb{R}^{N \times F \times T}$, where $N = \text{\#samples}$, $F = \text{\#features}$, $T = \text{time}$

Data Loading: After loading, each sample is

$X_i = \{x_i(t)\}_{t=1}^T \in \mathbb{R}^{F \times T}$ so firstly we map physical informations to a binary pirate-indicator variable $\text{isPirate} \in \{0, 1\}$, then, all continuous features are standardized using Z-score normalization, i.e. $\hat{x}_{i,f}(t) = \frac{x_{i,f}(t) - \mu_f}{\sigma_f}$. Finally, global feature extraction computes a fixed-length feature vector $g_i \in \mathbb{R}^D$ summarizing temporal dynamics, including statistical features (mean, variance, minimum, maximum, range) and domain-specific features (aggregated pain-survey and joint-activity statistics).

Additionally, when enabled PCA is applied independently to each trajectory $x_{i,f} = (x_{i,f}(1), \dots, x_{i,f}(T))^\top \in \mathbb{R}^T$. PCA yields $x_{i,f} \approx U_f z_{i,f}$, where U_f contains the leading eigenvectors and $z_{i,f}$ the corresponding POD coefficients. The number of retained components is the minimum k s.t. $\sum_{j=1}^k \lambda_{f,j} \geq 0.95 \sum_{j=1}^T \lambda_{f,j}$, with $\lambda_{f,j}$ once computed POD coefficients are appended to the global feature vector.

Global features are refined through a multi-stage selection procedure, including variance thresholding, removal of highly correlated features ($\rho > 0.95$), Random Forest feature-importance ranking, and Mutual Information scoring. The final subset is defined as the intersection of the top-ranked features across methods.

Additionally to address class imbalance the class weights are computed using the inverse-frequency scheme: $w_c = \frac{N}{C N_c}$, where $N_c = \#\{\text{samples} \in c\}$.

3.2 Pipeline

The Pipeline module provides a production-grade orchestration framework for end-to-end deep learning experiments with automated hyperparameter optimization. Its central component, the `FinalPipeline` class, performs architecture selection, K -fold cross-validation, checkpointing, distributed training, and submission generation. The system integrates Optuna with TPE sampler and Median Pruner, a DB for persistent experiment tracking, PyTorch Lightning for modular training and logging, and TensorBoard for monitoring. The module supports multiple macro-architectures made of modular micro-architectures.

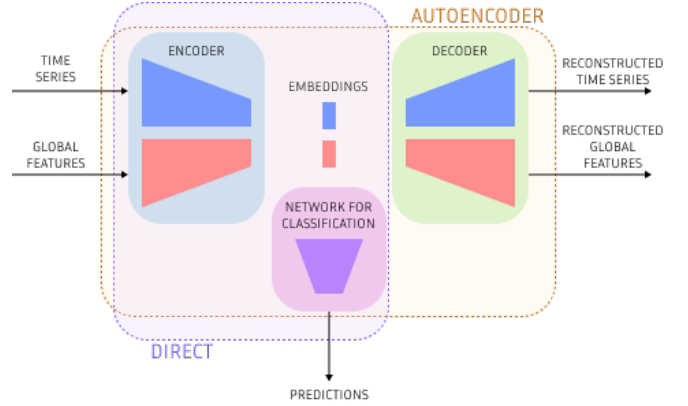
Given the datasets $X \in \mathbb{R}^{N \times F \times T}$, $G \in \mathbb{R}^{N \times D}$, the pipeline automates the selection of model parameters θ , trained via cross-validation to maximize a validation metric (mean F1-score). The optimization routine executes n_{trials} . For each trial j , Optuna samples a hyperparameter configuration $\theta^{(j)} \sim q(\theta \mid \mathcal{S})$, where q is the TPE-induced density esti-

mator. The objective function evaluates the metrics across K folds: $\mathcal{L}(\theta^{(j)}) = \frac{1}{K} \sum_{k=1}^K \text{F1}_{\text{val}}^{(k)}(\theta^{(j)})$.

Each fold involves the creation of training/ validation splits, construction of encoder/ decoder/ head modules according to $\theta^{(j)}$, application of He initialization, optional wrapping in a windowed model, and PyTorch Lightning training with early stopping and checkpointing.

The pipeline generate predictions $P \in \mathbb{R}^{N_{\text{test}} \times C}$, applies aggregation rules, converts class indices to textual labels, and writes the final CSV.

3.3 Architectures



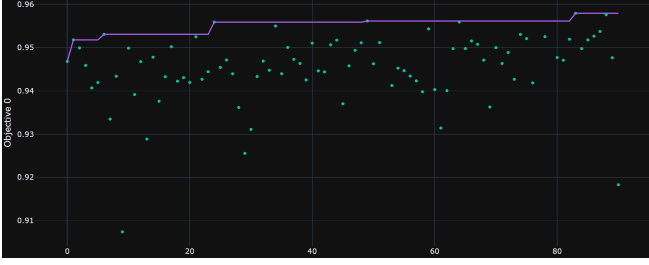
Supported architectures include RNN encoders (multi-layer LSTM/GRU with optional bidirectionality), Conv1D encoders (temporal convolutions with pooling), MultiScaleCNN (parallel convolutional branches at multiple receptive fields), and feedforward classification heads. Additionally if enabled we build a symmetric decoder.

4 Experiments

The highest F1 score we found came from the following process in the fully automated pipeline: for every trial generated by the Optuna study, we construct the encoder and, when enabled, a decoder for autoencoder-based models, apply He initialization, train over all K folds using PyTorch Lightning, perform model-level windowing, and aggregate predictions.

The best model is chosen using the macro-averaged F1 score, a metric particularly suitable for class-imbalanced multiclass problems as in our case.

$$\text{F1}_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C \text{F1}_c, \text{F1}_c = \frac{2 \text{Precision}_c \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}.$$



5 Results

Results

6 Discussion

The experimentation phase highlights that the most promising architecture is the autoencoder, thanks to its self-supervised properties, while at the micro-architecture level MultiScaleCNNs and recurrent layers both perform well because they capture local and multiscale temporal dependencies.

Data augmentations such as jittering, scaling, and time warping further improved robustness, although their impact was constrained by the limited dataset size. While global features also contributed meaningfully to performance—supported by a multi-layer embedding module—the classification of minority classes, particularly High Pain, remained challenging despite class-weighting strategies.

Overall, the system demonstrates solid generalization within the competition constraints, yet performance bottlenecks appear to be primarily linked

to the limited expressiveness of the data rather than architectural insufficiency.

7 Conclusions

This work presented a complete pipeline for pain-level classification from multivariate time series, integrating preprocessing, feature extraction, a MultiScaleCNN-based autoencoder architecture, extensive hyperparameter optimization, and robust evaluation via K-fold cross-validation.

The experimentation phase showed that the top-performing configuration combined multiscale temporal convolutions, a compact global-feature branch, and windowed prediction aggregation. This setup achieved strong overall performance and confirmed the effectiveness of modular temporal modeling. However, the evaluation also revealed persistent challenges, notably class imbalance and the difficulty of capturing distinctive patterns for the rarer pain levels, reflecting limitations primarily linked to the data rather than the architecture itself.

Future improvements could involve integrating attention-based encoders, experimenting with self-supervised temporal representations, or employing ensemble strategies to reduce variance. Expanding the dataset or augmenting it with synthetic but physiologically meaningful trajectories could further enhance performance, especially for underrepresented classes.

References